Schéma de la base de données - pili-pos

Conformément au §II.C du Manuel de certification SCE 2.0, ce document liste **chaque table**, **chaque champ**, son **type**, sa **longueur** (le cas échéant) et sa **description**.

Moteur : SQLite 3.x bundled via `rusqlite` (Rust). **Encodage** : UTF-8. **Mode** : WAL (Write-Ahead Logging) - atomique crash-safe. **Fichier vivant** : `/var/lib/pili-pos/pos.db` en production. **Source de vérité du schéma** : `crates/pos-db/src/schema.rs` (lecture machine via `sqlite3 pos.db ".schema"`).

1. Tables de référence

1.1 `languages`

Langues prises en charge par les traductions multi-langues.

Champ	Type	Longueur	Description
`code`	TEXT	2	Clé primaire. Code ISO 639-1 : `fr`, `nl`, `de`, `en`, `zh`
`name`	TEXT	-	Nom de la langue dans cette langue (“Français”, “Deutsch”, “中文”…)
`active`	INTEGER	1 (booléen)	1 = langue activée pour ce déploiement, 0 = désactivée

Données initiales : `fr`, `nl`, `en`, `zh` insérées par la migration `001` ; `de` ajouté par la migration `007` (§6) pour qu’un catalogue en région de langue allemande puisse porter des désignations en allemand (la FK `translations.lang` → `languages.code` les refusait sinon) - voir `crates/pos-db/src/migrate.rs`.

1.2 `translations`

Table polymorphe : associe une chaîne traduite à n’importe quelle entité de la base (catégorie, item, modificateur…).

Champ	Type	Description
`id`	INTEGER	Clé primaire auto-incrémentée
`entity_type`	TEXT	Type d’entité : `category`, `menu_item`, `modifier_group`, `modifier`
`entity_id`	INTEGER	ID de l’entité concernée
`field`	TEXT	Nom du champ traduit : typiquement `name`, parfois `description`
`lang`	TEXT (FK → `languages.code`)	Code langue de la traduction
`value`	TEXT	Texte traduit

Contrainte d’unicité : `UNIQUE(entity_type, entity_id, field, lang)`. Index : `idx_translations_lookup(entity_t
entity_id, lang)`.

1.3 `categories`

Catégories de menu (ex. “Sushi”, “Boissons”).

Champ	Type	Description
id	INTEGER	Clé primaire
external_id	TEXT (unique)	Référence externe (NULL en build cert)
sort_order	INTEGER	Ordre d'affichage
color	TEXT	Couleur hexadécimale optionnelle
active	INTEGER	Drapeau actif (0/1)

1.4 menu_items

Articles vendus. Chaque article appartient à exactement une catégorie et porte UN code TVA SCE 2.0 (A/B/C/D/X).

Champ	Type	Description
id	INTEGER	Clé primaire
external_id	TEXT (unique)	Référence externe (NULL en build cert)
category_id	INTEGER (FK → categories.id)	Catégorie parente
price_cents	INTEGER	Prix unitaire TVA incluse en centimes d'euro
vat_code	TEXT	Code TVA SCE en salle : A, B, C, D, X
vat_category	TEXT	Catégorie fiscale webshop : food, beverage, zero_rated, out_of_scope
vat_code_takeaway	TEXT (nullable, migration 002)	Code TVA SCE à emporter, quand il diffère du code en salle (ex. B→C) ; NULL = le code en salle s'applique aussi à l'emporté
open_price	INTEGER (migration 002)	1 = "prix libre" - l'opérateur saisit le prix unitaire à l'ajout (ex. produit 22010), 0 = prix fixe de price_cents
kitchen_printer	INTEGER	1 = à envoyer en cuisine, 0 = imprimante caisse seulement
sort_order	INTEGER	Ordre d'affichage dans la grille
active	INTEGER	1 = article disponible, 0 = retiré du menu

1.5 modifier_groups

Groupes de modificateurs liés à un article (ex. "Sauce" pour les sushi).

Champ	Type	Description
id	INTEGER	Clé primaire
external_id	TEXT (unique)	Référence externe
min_select	INTEGER	Nombre minimum de modificateurs à sélectionner
max_select	INTEGER	Nombre maximum
menu_item_id	INTEGER (FK → menu_items.id)	Article parent

1.6 `modifiers`

Options individuelles d'un groupe (ex. "Sauce soja").

Champ	Type	Description
`id`	INTEGER	Clé primaire
`external_id`	TEXT (unique)	Référence externe
`group_id`	INTEGER (FK → `modifier_groups.id`)	Groupe parent
`price_cents`	INTEGER	Supplément de prix (peut être 0)
`sort_order`	INTEGER	Ordre dans le groupe

2. Tables transactionnelles

2.1 `orders`

Commandes (dine-in et takeaway). Une commande peut être ouverte (`open`), fermée (`closed` - encaissée) ou annulée (`voided`).

Champ	Type	Description
`id`	INTEGER	Clé primaire
`table_number`	INTEGER (nullable)	Numéro de table pour les dine-in
`order_type`	TEXT	`dine_in` ou `takeaway`
`status`	TEXT	`open`, `closed` ou `voided`
`user_insz`	TEXT	INSZ de l'opérateur qui a ouvert la commande
`created_at`	TEXT (ISO 8601)	Date / heure d'ouverture
`closed_at`	TEXT (nullable)	Date / heure d'encaissement
`subtotal_cents`	INTEGER	Sous-total TVA incluse
`total_cents`	INTEGER	Total payé en centimes
`hubrise_order_id`	TEXT (nullable)	Identifiant HubRise d'une commande en ligne ; NULL pour une commande de la caisse
`is_fiscal`	INTEGER	1 = commande soumise au FDM ; une commande en ligne passe à 1 dès sa signature (P ou N)
`source`	TEXT	`pos_local` (caisse), ou canal de la commande en ligne (par exemple `deliveroo` ; `webshop` sans canal)
`hubrise_status`	TEXT (nullable)	Statut HubRise de la commande en ligne
`service_type`	TEXT (nullable)	Livraison, enlèvement ou sur place (commande en ligne)
`raw_hubrise_json`	TEXT (nullable)	Commande en ligne telle que reçue de HubRise

2.2 `order_lines`

Lignes d'une commande. Dénormalisées (le nom et le prix sont copiés depuis `menu_items` au moment de l'ajout, pour figer le tarif historique).

Champ	Type	Description
`id`	INTEGER	Clé primaire
`order_id`	INTEGER (FK → `orders.id`)	Commande parente

Champ	Type	Description
menu_item_id	INTEGER (FK → menu_items.id)	Article référencé
quantity	INTEGER	Quantité (peut être négative pour les remboursements)
unit_price_cents	INTEGER	Prix unitaire au moment de l'ajout (figé)
vat_code	TEXT	Code TVA appliqué (figé)
name	TEXT	Désignation imprimée (figée)
department_id	TEXT (nullable, migration 002)	Département SCE au moment de l'ajout, envoyé au FDM comme departmentId et totalisé dans les rapports X/Z
department_name	TEXT (nullable, migration 002)	Nom du département au moment de l'ajout, envoyé comme departmentName
modifiers_json	TEXT (nullable)	Modificateurs sélectionnés en JSON
created_at	TEXT	Date / heure d'ajout

order_lines reste la vue consolidée affichée/imprimée ; le journal chronologique des ajouts/corrections/remboursements est order_entries (§3.4).

2.3 payments

Paiements enregistrés pour une commande. Plusieurs paiements possibles par commande (espèces + carte sur un partage). Modes limités à cash, card, other (extensible - vouchers).

Champ	Type	Description
id	INTEGER	Clé primaire
order_id	INTEGER (FK → orders.id)	Commande payée
method	TEXT	cash, card ou other
amount_cents	INTEGER	Montant en centimes (positif)
created_at	TEXT	Date / heure du paiement

3. Tables fiscales (append-only)

3.1 fiscal_events Attention : APPEND-ONLY

Le cœur du système fiscal. Une ligne par événement SCE 2.0 envoyé au FDM, jamais modifiée ni supprimée après insertion.

Champ	Type	Description
id	INTEGER	Clé primaire auto-incrémentée
event_type	TEXT	N, P, T, C, I, F, S, ou R (contraint par CHECK)
sequence_number	INTEGER	Compteur local POS (cf. COUNTER_STRATEGY.md)
order_id	INTEGER (FK → orders.id, nullable)	Commande associée pour N/P/I/C ; NULL pour F/S/R
pos_id	TEXT	Identifiant POS 14 caractères (cf. spec POS v2.2)
terminal_id	TEXT	ID du terminal physique

Champ	Type	Description
user_insz	TEXT	INSZ de l'opérateur
event_data_json	TEXT	Payload SCE complet sérialisé en JSON
fdm_response_json	TEXT (nullable)	Réponse brute du FDM (signature, counters, footer, QR)
fdm_id	TEXT (nullable)	ID du FDM extrait de la réponse
fdm_datetime	TEXT (nullable)	fdmDateTime extrait
event_label	TEXT (nullable)	Label retourné par le FDM (peut différer de event_type en formation)
short_signature	TEXT (nullable)	40 caractères hexadécimaux (SHA-1), événements N uniquement, imprimés sur le ticket
qr_data	TEXT (nullable)	URL 38 caractères pour le QR code
created_at	TEXT	Date / heure d'insertion (= clôture de la tx)
archived	INTEGER	Vestige de schéma - toujours 0, candidat à suppression

Contrainte d'unicité : UNIQUE(event_type, sequence_number) - garantit la **stricte séquentialité par type d'événement**.

3.2 fiscal_counters

Compteurs persistants par label d'événement.

Champ	Type	Description
event_label	TEXT	Clé primaire = label SCE (N, P, T, C, I, F, S, R)
current_value	INTEGER	Dernier compteur émis pour ce label

Modification exclusive via pos_fiscal::counter::FiscalCounter::next (atomique avec l'archive, cf. COUNTER_STRATEGY.md).

3.3 z_reports

Index des rapports Z générés. Référence le fiscal_events correspondant.

Champ	Type	Description
id	INTEGER	Clé primaire
report_date	TEXT	Date de la journée fiscale clôturée
sequence_number	INTEGER	Numéro de Z séquentiel
data_json	TEXT	Snapshot des agrégats au moment du Z
fiscal_event_id	INTEGER (FK → fiscal_events.id)	Lien vers l'événement R signé
created_at	TEXT	Date / heure de génération

3.4 `order_entries` (migration `002`)

Journal append-only de chaque ajout, correction ou remboursement de ligne sur une commande ouverte (conforme POS v2.2 §2 “toutes les données d’entrée, par ordre chronologique, y compris les corrections”). `order_lines` reste la vue consolidée affichée/imprimée ; `order_entries` porte la trace chronologique transmise au FDM.

Champ	Type	Description
`id`	INTEGER	Clé primaire, ordre chronologique
`order_id`	INTEGER (FK → `orders.id`)	Commande parente
`line_id`	INTEGER	Ligne d’origine référencée
`menu_item_id`	INTEGER	Article
`quantity`	INTEGER (≠ 0)	Quantité de l’entrée (négative pour correction/remboursement)
`unit_price_cents`	INTEGER	Prix unitaire au moment de l’entrée
`vat_code`	TEXT	Code TVA (contraint A/B/C/D/X)
`name`, `department_id`, `department_name`	TEXT	Désignation et département au moment de l’entrée
`kind`	TEXT	`entry`, `correction` ou `refund` (contraint)
`sent_event_id`	INTEGER (FK → `fiscal_events.id`, nullable)	Rempli quand l’entrée a été transmise dans un événement P ou dans le N final
`created_at`	TEXT	Date / heure d’ajout

Deux triggers SQL font respecter l’append-only :

- `order_entries_no_delete` - `BEFORE DELETE`, avorte systématiquement (`RAISE(ABORT, ...)`).
- `order_entries_no_rewrite` - `BEFORE UPDATE` sur toutes les colonnes journalisées (`order_id`, `line_id`, `menu_item_id`, `quantity`, `unit_price_cents`, `vat_code`, `name`, `department_id`, `department_name`, `kind`, `created_at`), avorte systématiquement. Seule `sent_event_id` reste modifiable - elle passe de `NULL` à l’`id` de l’événement fiscal une fois l’entrée effectivement transmise au FDM, sans quoi elle ne serait jamais renseignable.

Index : `idx_order_entries_order(order_id, id)`.

3.5 `booking_periods` (migration `003`)

Persistance de la période d’ouverture (`bookingPeriodId`, POS v2.2 §2.2.2). Une période court du premier événement suivant un Z jusqu’au Z suivant, et porte la date d’ouverture (fuseau Europe/Brussels). Voir §III.J et §III.L de `REPONSES_MANUEL_CERT_FR.md`.

Champ	Type	Description
`id`	TEXT	Clé primaire (UUID v4, générée par `commands::booking_period`)
`booking_date`	TEXT	Date d’ouverture, fuseau Europe/Brussels
`opened_at`	TEXT	Horodatage d’ouverture
`closed_at`	TEXT (nullable)	Horodatage de clôture (NULL = période ouverte)
`closing_event_id`	INTEGER (FK → `fiscal_events.id`, nullable)	Événement R qui a clôturé la période (NULL tant qu’ouverte)

Contrainte `CHECK ((closed_at IS NULL) = (closing_event_id IS NULL))` - les deux colonnes sont renseignées ou nulles ensemble. Au plus une période est ouverte à la fois, via l’index unique partiel

idx_booking_periods_one_open sur l'expression (closed_at IS NULL) filtrée WHERE closed_at IS NULL (deux lignes ouvertes collisionneraient sur la même valeur d'expression). Deux triggers :

- booking_periods_no_reopen - BEFORE UPDATE ... WHEN OLD.closed_at IS NOT NULL, avorte toute modification d'une période déjà close (pas seulement sa réouverture).
- booking_periods_no_delete - BEFORE DELETE, avorte systématiquement.

3.6 ticket_deliveries Attention : APPEND-ONLY (partiel)

File d'envoi et journal d'audit du ticket numérique par e-mail (migration 005, cf. §III.H de REPONSES_MANUEL_CERT_FR.md et le détail complet dans TICKET_NUMERIQUE_FR.md). Une ligne par ticket dont le support choisi inclut l'e-mail (DIGITAL ou PAPER_DIGITAL), ajoutée dans la **même transaction** que l'événement fiscal qu'elle accompagne.

Champ	Type	Description
id	INTEGER	Clé primaire
fiscal_event_id	INTEGER (FK → fiscal_events.id)	Événement dont ce ticket est la copie numérique. **Immuable** (trigger).
medium	TEXT	DIGITAL (e-mail seul, papier en secours) ou PAPER_DIGITAL (papier toujours + e-mail en bonus). Contraint. **Immuable**.
recipient	TEXT (nullable)	Adresse e-mail en clair. NULL dès l'envoi réussi ou le repli papier, et au plus tard 72 h après création (purge RGPD).
recipient_hint	TEXT	Adresse masquée (ex. j***@e***.be), conservée indéfiniment pour la lisibilité du support/dossier.
ticket_text	TEXT	Texte du ticket, identique à ce qui s'imprime. **Immuable**.
verification_url	TEXT (nullable)	URL de vérification FDM ; NULL pour les copies (C) et tickets de formation (T), qui n'en reçoivent pas.
status	TEXT	pending, sending, sent, failed, fallback_printed, ou purged. Contraint.
attempts	INTEGER	Nombre de tentatives d'envoi effectuées
next_attempt_at	TEXT (nullable)	Prochaine échéance de réessai (recul exponentiel)
provider_message_id	TEXT (nullable)	Identifiant renvoyé par Scaleway TEM sur succès
last_error	TEXT (nullable)	Dernière erreur d'envoi
fallback_printed	INTEGER	1 = le papier de secours a été imprimé pour ce ticket
created_at / updated_at	TEXT	Horodatages

Triggers : suppression interdite, et modification interdite de fiscal_event_id / ticket_text / medium (les autres colonnes, notamment status, restent modifiables - c'est le cycle de vie de la file). Index : idx_ticket_deliveries_due(status, next_attempt_at) pour le worker de réessai.

3.7 `original_prints` Attention : APPEND-ONLY

Une ligne par événement fiscal dont l'**original** a été imprimé (migration `006`, POS spec v2.2 §8.1 : un original n'est imprimé qu'une seule fois). Sert de garde avant toute impression papier d'origine (checkout, remboursement, ou rattrapage d'un envoi numérique manqué) ; toute reproduction ultérieure passe par l'événement C (copie).

Champ	Type	Description
`fiscal_event_id`	INTEGER (FK → `fiscal_events.id`)	Clé primaire - au plus une ligne par événement
`printed_at`	TEXT	Horodatage de l'impression

Triggers : suppression et modification interdites (table strictement append-only, sans exception).

3.8 `users`

Utilisateurs du POS. INSZ comme clé primaire. Voir `PROFILS_UTILISATEURS_FR.md`.

Champ	Type	Description
`insz`	TEXT (11 chiffres)	Clé primaire. Numéro National Belge
`name`	TEXT	Nom affichable
`role`	TEXT	`admin`, `cashier`, ou `waiter` (contraint)
`pin_hash`	TEXT (nullable)	Hash bcrypt du PIN optionnel
`active`	INTEGER	1 = utilisateur actif, 0 = désactivé

4. Tables Peppol (build cert)

4.1 `peppol_deliveries`

Audit trail des factures B2B livrées via Access Point Peppol.

Champ	Type	Description
`id`	INTEGER	Clé primaire
`fiscal_event_id`	INTEGER (FK)	Lien vers l'événement I correspondant
`invoice_no`	TEXT	Numéro de facture émetteur
`customer_endpoint_id`	TEXT	Identifiant Peppol du destinataire (`0208:BE...`)
`ap_vendor`	TEXT	Nom du fournisseur AP ("storecove", "unifiedpost", "logging-only")
`ap_message_id`	TEXT (nullable)	ID retourné par l'AP
`ubl_xml`	TEXT	Document UBL 2.1 complet pour audit
`ap_response_json`	TEXT (nullable)	Réponse de l'AP
`status`	TEXT	`queued`, `accepted`, ou `failed` (contraint)
`error_msg`	TEXT (nullable)	Message d'erreur si `failed`
`created_at`	TEXT	Date / heure de l'envoi
`accepted_at`	TEXT (nullable)	Date / heure d'acceptation par l'AP

Index : `idx_peppol_deliveries_invoice(invoice_no)`.

5. Tables des commandes en ligne (migrations 20 à 25)

Présentes sur toute installation ; écrites seulement quand les commandes en ligne sont activées pour l'installation (COMMANDES_EN_LIGNE_FR.md §2), sauf le journal de l'interrupteur.

5.1 webshop_orders

Une ligne par commande en ligne, à côté de sa ligne orders.

Colonne	Type	Description
order_id	INTEGER PK	Référence orders(id)
hubrise_order_id	TEXT UNIQUE	Identifiant HubRise ; reference du costCenter
channel / channel_kind	TEXT	Canal (par exemple deliveroo) ; platform ou webshop
service_type	TEXT	delivery, collection ou eat_in
payment_mode / payment_type	TEXT	prepaid ou unpaid ; APP ou ONLINE pour une commande payée
fiscal_state	TEXT	pending, signing, signed_p, signed_n, voided, rejected, refund_pending, cancel_pending, refunded, closed
resolution	TEXT	ok ou needs_review (« à vérifier »)
attention_json / warnings_json	TEXT (JSON)	Raisons du contrôle, avertissements
lines_json	TEXT (JSON)	Lignes à signer, rapprochées du menu
p_event_id / n_event_id / refund_event_id	INTEGER	Référencent fiscal_events(id)
accepted_by / accepted_at	TEXT	INSZ de l'employé qui a accepté, date

Index : idx_webshop_orders_state(fiscal_state, resolution).

5.2 hubrise_channels, hubrise_cash_payment_refs

hubrise_channels associe un canal à son type (platform / webshop), au type de paiement FDM d'une commande payée (APP / ONLINE) et au traitement de ses frais (charge_policy : include ou exclude, exclude par défaut pour les plateformes depuis la migration 24). hubrise_cash_payment_refs liste les références de paiement HubRise qui désignent des espèces.

5.3 order_price_changes (ajout seul)

Remises d'une commande en ligne, envoyées au FDM comme priceChanges (spécification POS v2.2). Une ligne par remise et par ligne du journal (entry_id référence order_entries(id)) : change_id, name, scope (LINE / EVENT), change_type (PUBLIC / INTERNAL), amount_cents. Déclencheurs refusant UPDATE et DELETE.

5.4 integration_switches, integration_switch_log (ajout seul)

integration_switches : état de l'interrupteur (name = hubrise, enabled, config_value = valeur de pos.json au dernier démarrage, changed_at, changed_by). integration_switch_log : chaque change-

ment (enabled, source = config ou admin, changed_by = INSZ du Gérant, changed_at). Déclencheurs refusant UPDATE et DELETE.

5.5 Liaison HubRise et menu reflet

hubrise_connection (connexion à l'établissement HubRise, état du rappel, dernière erreur), hubrise_events (événements reçus, nombre de tentatives), hubrise_outbox (statuts à envoyer à HubRise, dans l'ordre par commande) ; catalog_sync_state (dernier catalogue appliqué ou refusé, verrou du menu). La migration 21 ajoute menu_items.kind (product / charge), menu_items.on_pos, categories.hidden et active / sort_order sur les groupes d'options et les options.

6. Migrations

Système de migrations versionné dans crates/pos-db/src/migrate.rs. À la version actuelle, **quinze migrations** existent (1 à 9, puis 20 à 25 ; 10 à 19 sont réservées aux correctifs du binaire certifié), appliquées dans l'ordre et consignées dans la table _migrations_applied (version, name, applied_at) :

Version	Nom	Contenu
1	initial-schema	Crée l'ensemble des tables listées au §§1-5 ci-dessus.
2	catalogue-fiscal-fields	Ajoute vat_code_takeaway et open_price sur menu_items, department_id/department_name sur order_lines, et la table order_entries avec ses triggers (§3.4).
3	booking-periods	Ajoute la table booking_periods avec ses triggers et son index (§3.5).
4	fiscal-archive-append-only	Ajoute les triggers interdisant DELETE/UPDATE sur fiscal_events et z_reports (§3.1, §3.3).
5	ticket-deliveries	Ajoute la table ticket_deliveries (§3.6, ticket numérique par e-mail).
6	original-prints	Ajoute la table original_prints (§3.7, impression unique de l'original).
7	german-language	Insère de (Deutsch) dans languages, pour qu'un catalogue en région de langue allemande puisse porter des désignations en allemand (§1.1).
8	meal-voucher-payments	Autorise le moyen de paiement meal_voucher dans payments (table reconstruite avec ses lignes, SQLite ne modifiant pas un CHECK).
9	shifts-and-pre-bill	Ajoute staff_shifts (index local des prestations ouvertes par un S WorkIn) et orders.pre_bill_at.

Version	Nom	Contenu
20	`hubrise-transport`	Colonnes de nouvelle tentative et d'erreur sur `hubrise_connection`, `hubrise_events`, `hubrise_outbox` (§5.5).
21	`catalog-sync`	Menu reflet d'un catalogue externe, `catalog_sync_state` (§5.5).
22	`webshop-orders`	`webshop_orders`, `hubrise_channels`, `hubrise_cash_payment_refs` (§5.1, §5.2).
23	`price-changes`	`order_price_changes` et ses déclencheurs (§5.3).
24	`platform-charges`	Frais des plateformes exclus par défaut (§5.2).
25	`integration-switch`	`integration_switches`, `integration_switch_log` et ses déclencheurs (§5.4).

Chaque migration s'exécute dans sa propre transaction `BEGIN IMMEDIATE` ; un échec entraîne un `ROLLBACK` complet de cette migration sans toucher les précédentes déjà consignées.

Les bases "legacy" créées avant la versification sont automatiquement annotées comme `v1 (backfilled)`.

7. Méthode d'export pour audit

Voir `FEUILLE_ACQUISITION_DONNEES_FR.md` §2.